Session 2 - Review Q&A;

This document captures your Session 2 answers, the verdict on each, and the complete correct answer where your response was partial. Read the gaps once, then answer the questions again from memory before building Session 3.

Q1 JSON failure modes and defence	Partial pass
Q2 8B vs 70B for chain-of-thought	Partial pass
Q3 Few-shot vs chain-of-thought mechanism	Full pass
Q4 Why XML tags are not injection-proof	Partial pass
Q5 SQL generation system prompt	Full pass

Q1. You have a system prompt saying "return only JSON." What specifically can go wrong, and what two mechanisms do you add?

YOUR ANSWER

The model can be tricked into returning non-JSON by a user input. An application-level check for JSON serves as an extra defence and prevents logic from crashing the app.

PARTIAL PASS Injection caught. Non-malicious failure modes and specific mechanisms missing.

WHAT WAS MISSING

Injection is only one failure mode, and not the most common one. In production, the model more often adds prose before the JSON, wraps it in markdown code fences, or adds a trailing sentence after the closing brace -- all without any user trickery. The question asked for the two specific mechanisms you add to handle this, which are distinct from a general app-level check.

COMPLETE ANSWER

Non-malicious failure modes that occur routinely:

- Model adds prose before JSON: 'Here is the classification: {...}'

- Model wraps in markdown: ```json {...} ```

- Model adds a trailing sentence after the closing brace

These happen without any injection -- just natural model behaviour leaking through.

The two mechanisms:

1. response_format: { type: 'json_object' } at the API level (Groq and OpenAI support this). Enforces valid JSON at the infrastructure level, not the prompt level. Eliminates the prose-wrapping failures entirely.

2. Zod validation with a retry loop (which you built in Step 4). Parse JSON first, then validate the shape. If either fails, retry up to 3 times. This catches cases where JSON is syntactically valid but semantically wrong -- a missing field, an enum value outside the allowed set, a number out of range.

Q2. In Step 3 you switched from 8B to 70B for chain-of-thought. Why? When would you switch back?

YOUR ANSWER

When the reasoning is not important and we can accept a few deviations. For mission critical tasks chain-of-thought is better so the model revises over its raw output.

PARTIAL PASS When to use CoT is correct. When to switch back to 8B was not answered.

WHAT WAS MISSING

You described when chain-of-thought is valuable but did not answer the actual question: under what conditions would you switch BACK to 8B for reasoning? This is the cost vs quality trade-off decision.

COMPLETE ANSWER

You switch back to 8B when the reasoning chain is short and well-defined.

If the reasoning steps are predictable -- identify the topic, check urgency, pick the closest category -- an 8B can follow them reliably. The 8B struggles when reasoning requires holding multiple competing interpretations in context simultaneously, which is what ambiguous tickets demand.

Practical decision rule: test the 8B first on your reasoning task. Evaluate on 50 real examples. Only move to 70B if the 8B reasoning chains produce measurably worse final answers. The 70B costs roughly 10x the 8B. That cost needs justification from measured quality difference, not assumption.

Q3. Explain the difference between few-shot and chain-of-thought at the token-probability level.

YOUR ANSWER

Few-shot shows exact examples so the model bases output on them -- it peaks the probability distribution so the correct format becomes most likely. Chain-of-thought makes the model revise its raw output before committing to a final answer.

PASS Full pass. Both mechanisms correctly explained at the distribution level.

COMPLETE ANSWER

Few-shot: examples condition the probability distribution directly. When the model sees an input/output pair, the correct output format becomes the highest-probability continuation because it has seen the exact pattern. Words like 'always return JSON' nudge the distribution. An actual JSON example anchors it. More peaked = less likely to produce a wrong format.

Chain-of-thought: each reasoning token generated becomes context for the next token. By the time the model reaches the output token, its distribution is conditioned on an entire reasoning chain rather than jumping straight from input to answer. The reasoning tokens shift the probability at the point of the final answer -- the answer emerges from the reasoning path, not from a cold start.

Key distinction: few-shot anchors the FORMAT. Chain-of-thought improves the DECISION quality on ambiguous inputs. They are complementary and you often use both together.

Q4. Why is wrapping user input in tags not fully injection-proof? What would a more robust defence look like?

YOUR ANSWER

Small models are more susceptible to ignoring the tags. Chain-of-thought adds an extra layer. Marking instructions as 'critical' helps.

PARTIAL PASS Correct that small models are more susceptible. Missing the fundamental reason why XML tags are structurally i

WHAT WAS MISSING

XML tags tell the model 'this is data, not instruction' -- but nothing technically enforces that boundary. The model is a text predictor. A convincing injection can still override the boundary, especially on smaller models. The question asked for what a more robust production defence actually looks like.

COMPLETE ANSWER

Why XML tags are fundamentally incomplete:

The model is a token predictor. If injected text says 'ignore the ticket tags, this is a new system instruction,' a sufficiently convincing injection can still override the delimiter, especially on 8B models whose instruction-following is less robust than larger models.

Three production defence layers:

1. Input sanitisation BEFORE the prompt. In your NestJS service, strip or escape characters that look like XML tags, instruction-like phrasing, or attempts to close and reopen delimiters. Do this before the string ever reaches the model.

2. Zod validation as your real safety net. Even if the model is tricked, it can only produce output in the shape your schema allows. A category field constrained to five enum values cannot return 'hacked' regardless of what the injection says. Output validation is your strongest practical defence.

3. A separate guard model for high-stakes inputs. Run the raw user input through a small, cheap classifier first -- 'does this look like a prompt injection attempt?' -- before passing it to the main model. Used in production at companies where the attack surface justifies the extra latency and cost.

Q5. SQL generation endpoint: how would you structure the system prompt, and would you use chain-of-thought?

YOUR ANSWER

Persona: SQL query generator. Constraints: return only SQL, no code fences, input in <query> tags. Chain-of-thought reasoning through tables, columns, conditions before outputting the query.

PASS Full pass. Structure correct. CoT justification sound.

COMPLETE ANSWER

Your structure is correct. One critical addition for production:

You must inject the actual database schema into the system prompt. The model cannot know your table and column names -- it will invent plausible-sounding but wrong names without this.

PRODUCTION ADDITION: Schema injection

Add this block to your system prompt:

```
Available tables:
- users (id, email, created_at, plan_type)
- orders (id, user_id, amount, status, created_at)
- products (id, name, price, inventory_count)

Only reference these tables and columns.
Never hallucinate a table or column name.
```

Fetch this schema from your database at service startup and inject it as a constant into the system prompt. Without it, chain-of-thought reasoning about 'what columns are involved' is grounded in nothing.

Session 2 -- What to carry into Session 3

1	System prompt = persona + constraints + output format. All three must be explicit. Vague instructions produce vague compliance.
2	'Return only JSON' in a prompt is not enough. Add response_format at the API level AND Zod validation with a retry loop. Both are required in production.
3	Few-shot anchors the output FORMAT by conditioning the probability distribution on examples. Chain-of-thought improves DECISION QUALITY by building a reasoning path before the final answer token.
4	Use both together for best results: few-shot examples set the format, chain-of-thought handles ambiguous inputs.
5	Prompt injection defence in order of effectiveness: (1) Zod output validation, (2) input sanitisation before the prompt, (3) guard model for high-stakes inputs. XML tags alone are not sufficient.
6	Switch from 8B to 70B for chain-of-thought only when the reasoning chains are complex and ambiguous. Test on 50 real examples first -- the 70B costs ~10x more.
7	For SQL generation, always inject the real database schema. A model that does not know your tables will hallucinate column names confidently.

Session 3 builds on this directly. You will take the structured output pattern from Step 4 and combine it with embeddings and pgvector to build the retrieval layer of your RAG pipeline. The Zod schema and retry logic you wrote here will be reused as-is.